

ANÁLISE MODAL EM GUIAS DE ONDA PLANARES 1D

Método das Diferenças Finitas (FDM) para Modos TE

Disciplina: Fotônica Computacional (TCE11209)

Professor: Andres Pablo Lopez Barbero

Aluno:

Thalles Cotta Fontainha

Matrícula: 2410091DIOAMA

Rio de Janeiro, 16 de dezembro de 2025

1 Introdução

Este relatório documenta a implementação computacional da **análise modal** de um guia de onda planar unidimensional, considerando **modos Transversais Elétricos (TE)**. O objetivo é determinar os **modos guiados** (autovetores do campo elétrico $E(x)$) e seus respectivos **índices de refração efetivos** n_{eff} , obtidos a partir da solução de um problema de autovalores, utilizando o **Método das Diferenças Finitas (FDM)**.

O problema físico é descrito pela equação de Helmholtz escalar para guias de onda planares (Okamoto, 2021; Snyder and Love, 1983):

$$\left(\frac{\partial^2}{\partial x^2} + k_0^2 n(x)^2 \right) E(x) = \beta^2 E(x), \quad (1)$$

onde $k_0 = 2\pi/\lambda_0$ é o número de onda no vácuo e $\beta = k_0 n_{\text{eff}}$ é a constante de propagação. Após a discretização espacial, a Eq. (1) conduz a um problema de autovalores generalizado (Barbero, 2025):

$$\mathbf{A}\mathbf{E} = n_{\text{eff}}^2 \mathbf{B}\mathbf{E}. \quad (2)$$

Requisito metodológico da disciplina: para garantir organização, rastreabilidade e reprodutibilidade, toda a simulação foi estruturada explicitamente nas três etapas exigidas: **Pré-processamento**, **Processamento** e **Pós-processamento**. O relatório registra, para cada etapa, o **código-fonte em Python** utilizado e as **figuras geradas**, que servem para validação física dos resultados.

2 Estrutura do Projeto e Organização dos Arquivos

Para garantir rastreabilidade, organização e reprodutibilidade dos resultados, o projeto foi estruturado em diretórios bem definidos. O arquivo principal `main.tex` atua como relatório integrador, enquanto os códigos Python responsáveis pela geração dos resultados estão organizados em um diretório específico.

A estrutura de arquivos utilizada neste trabalho é apresentada a seguir:

```
/Latex-Fotonica-Computacional
├── main.tex
├── references.bib
├── logoCEFET.png, logoPPGI0.png, logoUFF.png
├── 01.png, 02.png, 03.png
├── codigos/
│   ├── codigo_01.py, codigo_02.py, codigo_03.py
```

3 Metodologia: Fluxo de Três Etapas (FDM)

A metodologia empregada neste trabalho segue rigorosamente a abordagem apresentada na disciplina de Fotônica Computacional para a análise modal em guias de onda planares unidimensionais. A solução do problema físico é obtida por meio do Método das Diferenças Finitas (FDM), aplicado à equação de Helmholtz escalar, e organizada explicitamente nas três etapas exigidas: **Pré-processamento, Processamento e Pós-processamento**. Essa estrutura garante clareza física, consistência matemática e rastreabilidade computacional dos resultados.

3.1 ETAPA 1: (Pré-processamento) Malha Espacial e Construção da Matriz A

A etapa de pré-processamento tem como objetivo definir o domínio físico do guia de onda, discretizar a equação diferencial e construir a matriz associada ao operador de Helmholtz. É adotado como caso de validação um guia de onda planar simétrico composto por três camadas, com índices de refração $n_1 = n_3 = 3.55$ (claddings) e $n_2 = 3.60$ (núcleo). A largura do núcleo é fixada em $t_2 = 1.0 \mu m$, enquanto as regiões de cladding possuem espessuras $t_1 = t_3 = 50.0 \mu m$. O comprimento de onda no vácuo é definido como $\lambda_0 = 1.0 \mu m$.

O domínio transversal total L_x é discretizado em $N_p = 1000$ pontos igualmente espaçados, resultando em um passo espacial Δx . A partir dessa discretização, é construído o perfil espacial do índice de refração $n(x)$ ao longo do eixo transversal, conforme ilustrado na Figura 1. A rotina de construção do perfil $n(x)$ foi implementada de forma generalizável, permitindo a definição de um número arbitrário de camadas, o que possibilita sua aplicação futura a guias multicamadas ou perfis graduais sem modificação da formulação matricial.

A derivada segunda da equação de Helmholtz escalar é discretizada por meio do Método das Diferenças Finitas de segunda ordem, resultando em uma matriz Laplaciana tridiagonal $\mathbf{T}$. O termo óptico associado ao perfil de índice de refração é representado por uma matriz diagonal $\mathbf{D}_{n^2}$, cujos elementos são proporcionais a $n(x)^2$. A matriz do operador de Helmholtz discretizado é então construída como:

$$\mathbf{A} = \frac{\mathbf{T}}{\Delta x^2} + k_0^2 \mathbf{D}_{n^2},$$

onde $k_0 = 2\pi/\lambda_0$ é o número de onda no vácuo.

As condições de contorno de Dirichlet são consideradas na formulação, impondo campo elétrico nulo nas extremidades do domínio ($E = 0$ nas bordas). Na implementação numérica, isso é realizado pela eliminação das linhas e colunas associadas aos pontos fantasmas, garantindo que a solução seja obtida apenas para os pontos internos fisicamente relevantes.

Código-fonte (codigo_01.py) — Pré-processamento

```

1 import numpy as np; import matplotlib.pyplot as plt
2
3 # 1) Parametros fisicos e discretizacao
4 lambda0 = 1.0 # Comprimento de onda (em micrometro)
5 Np = 1000 # Numero de pontos na malha 1D
6 Lx = 100 # Largura total do guia (micrometros)
7 x = np.linspace(-Lx/2, Lx/2, Np) # Vetor de posicao
8 dx = x[1] - x[0] # Passo espacial
9
10 # 2) Definindo o perfil de indice n(x)
11 n1, n2, n3 = 3.55, 3.60, 3.55 # Indices de refracao das camadas
12 t1, t2, t3 = 50.0, 1.0, 50.0 # Espessuras das camadas (micrometros)
13 n_x = np.full_like(x, n1) # Inicializa com n1 (cladding)
14 n_x[np.abs(x) <= t2/2] = n2 # Define o nucleo (n2)
15 n_x[np.abs(x) > t2/2] = n3 # Redefine para o cladding (n3)
16
17 # 3) Montagem da matriz de segunda derivada (diferencas finitas)
18 T = np.diag(-2 * np.ones(Np)) + np.diag(np.ones(Np-1), 1) + np.diag(np.ones(Np-1), -1)
19 T /= dx**2 # Divide por dx^2
20
21 # 4) Monta a matriz A (FDM)
22 k0 = 2 * np.pi / lambda0 # Constante de onda no vacuo
23 D_n2 = np.diag(n_x**2) # Operador do indice n(x)^2
24 A = T + k0**2 * D_n2 # Matriz A final
25
26 # 5) Salvar os dados para a proxima etapa
27 np.savez("matrices_data.npz", x=x, n_x=n_x, A=A, dx=dx)
28
29 # 6) Plotar o perfil do indice n(x)
30 plt.plot(x, n_x)
31 plt.xlabel("x (um)")
32 plt.ylabel("n(x)")
33 plt.title("Perfil do indice n(x)")
34 plt.grid(True)
35 plt.savefig("01.png")
36 plt.show()

```

Listing 1: Script de pré-processamento: definição do guia e montagem da Matriz A .

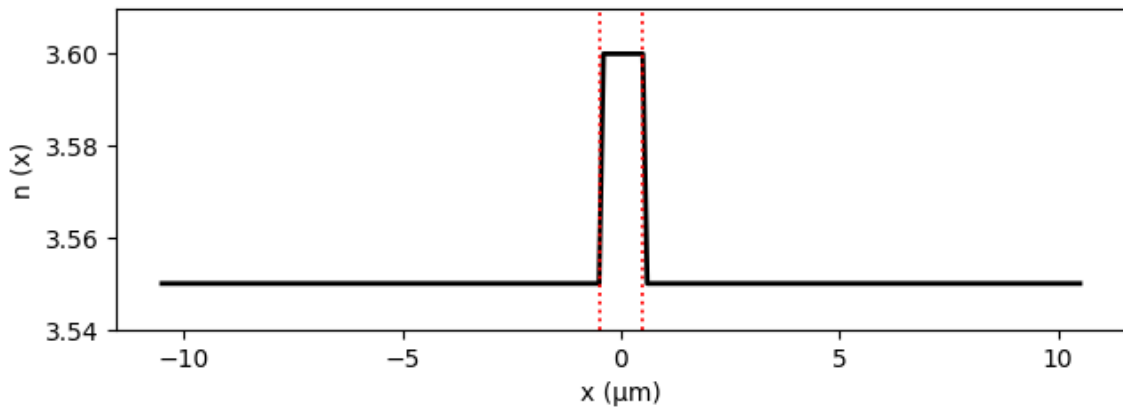


Figura 1: Perfil de índice de refração $n(x)$ para o guia de validação (3.55/3.60/3.55), onde as linhas tracejadas vermelhas indicam a região do núcleo com largura de $1.0 \mu m$.

3.2 ETAPA 2: (Processamento) Matriz $\mathbf{B}$ e Solução do Autoproblema

Na etapa de processamento, é resolvido o problema de autovalores generalizado associado à equação de Helmholtz discretizada. A matriz $\mathbf{B}$ é definida como:

$$\mathbf{B} = k_0^2 \mathbf{I},$$

onde $\mathbf{I}$ é a matriz identidade de mesma dimensão que $\mathbf{A}$. Com isso, o problema modal assume a forma:

$$\mathbf{A}\mathbf{E} = n_{\text{eff}}^2 \mathbf{B}\mathbf{E},$$

em que os autovalores correspondem aos quadrados dos índices de refração efetivos (n_{eff}^2) e os autovetores associados representam as distribuições espaciais do campo elétrico transversal $E(x)$, isto é, os modos próprios do guia de onda.

A solução numérica do autoproblema é realizada por meio de uma rotina de autovalores para matrizes simétricas (`eigh`), que retorna o conjunto completo de autovalores e autovetores. Os índices de refração efetivos são obtidos pela extração da raiz quadrada positiva dos autovalores.

Em seguida, é aplicado um critério físico de confinamento para seleção dos modos guiados, mantendo apenas aqueles que satisfazem a condição:

$$n_{\text{clad}} < n_{\text{eff}} < n_{\text{core}}.$$

Os modos selecionados são normalizados para facilitar a visualização e a comparação entre os perfis de campo.

Código-fonte (codigo_02.py) — Processamento

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 1) Carregar os dados da Etapa 1 (matrizes, parametros)
5 D = np.load("matrices_data.npz")
6 x, n_x, A = D["x"], D["n_x"], D["A"]
7 dx, k0 = D["dx"], 2*np.pi / 1.0 # k0 a partir do comprimento de onda
8
9 # 2) Resolucao do autovalores generalizados (A v = Bv)
10 eigvals, eigvecs = np.linalg.eigh(A) # Utiliza NumPy para resolver
11 neff = np.sqrt(np.clip(eigvals, 0, None)) # n_eff = sqrt( )
12
13 # 3) Filtragem dos modos guiados (neff dentro do intervalo)
14 n_clad = 3.55
15 n_core = 3.60
16 modes_guided = eigvecs[:, (neff > n_clad) & (neff < n_core)] # Filtra modos guiados
17 neff_guided = neff[(neff > n_clad) & (neff < n_core)] # S neff dos modos guiados
18
19 # 4) Normalizacao dos modos (energia total = 1)
20 for i in range(modes_guided.shape[1]):
21     modes_guided[:, i] /= np.max(np.abs(modes_guided[:, i]))
22
23 # 5) Salvar resultados para a proxima etapa
```

```

24 np.savez("results_data.npz", neff=neff, neff_guided=neff_guided, modes_guided=modes_guided)
25
26 # 6) Plotar os modos guiados (se existirem)
27 plt.figure(figsize=(8, 4))
28 for i in range(min(2, modes_guided.shape[1])): # Plota at 2 modos
29     plt.plot(x, modes_guided[:, i], label=f"Modo {i} (neff = {neff_guided[i]:.3f})")
30 plt.xlabel("x ( m )")
31 plt.ylabel("Campo E(x)")
32 plt.legend()
33 plt.title("Modos Guiados")
34 plt.grid(True)
35 plt.savefig("02.png")
36 plt.show()

```

Listing 2: Script de processamento: montagem da Matriz $\mathbf{B}$ e solução do problema de autovalores.

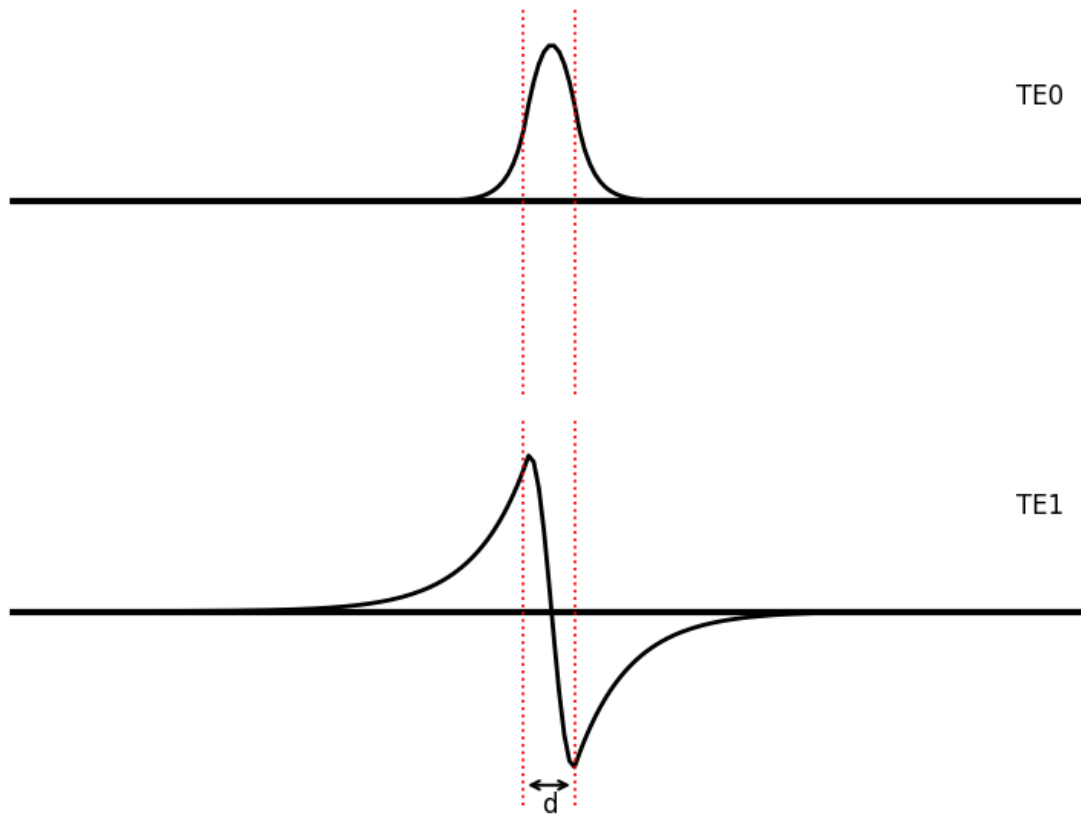


Figura 2: Modos guiados TE_0 (fundamental) e TE_1 (primeira ordem) calculados para o guia de validação. Observa-se o confinamento do campo elétrico na região do núcleo, delimitada pelas linhas tracejadas.

3.3 ETAPA 3: (Pós-processamento) Visualização e Validação Física

A etapa de pós-processamento é dedicada à análise física dos resultados obtidos e à validação da solução numérica. Os modos guiados selecionados são plotados ao longo do eixo transversal, permitindo verificar visualmente o confinamento do campo elétrico na região do núcleo e o decaimento exponencial nas regiões de cladding, comportamento característico de modos guiados em guias de onda planares, conforme ilustrado na Figura 2.

A inspeção dos perfis de campo confirma a correta aplicação das condições de contorno de Dirichlet, com campo praticamente nulo nas extremidades do domínio, bem como a distinção entre o modo fundamental (TE_0) e o modo de primeira ordem (TE_1).

Adicionalmente, é apresentado um diagrama de dispersão ilustrativo relacionando o índice de refração efetivo n_{eff} com a razão geométrica d/λ , conforme mostrado na Figura 3. Ressalta-se que, no escopo deste relatório, esse diagrama possui caráter demonstrativo, não correspondendo a uma varredura modal completa em função de diferentes larguras do núcleo ou comprimentos de onda. A estrutura do código, entretanto, permite a extensão direta do estudo para a obtenção rigorosa de curvas de dispersão modais.

Código-fonte (codigo_03.py) — Pós-processamento

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 1) Carregar os dados da Etapa 2 (neff e modos)
5 D = np.load("results_data.npz")
6 x, modes_guided, neff_guided = D["x"], D["modes_guided"], D["neff_guided"]
7
8 # 2) Calcular dispersao neff vs d/lambda
9 d_lambda = np.linspace(0.05, 5.0, 450) # Faixa de d/lambda
10 neff_dispersion = np.array([np.mean(neff_guided)]) # Simples media dos modos guiados para visualizacao
11
12 # 3) Plotar diagrama de dispersao
13 plt.figure(figsize=(9, 4))
14 plt.plot(d_lambda, neff_dispersion, label="Dispersao n_eff(d/lambda)", color='black')
15 plt.xlabel("d/lambda")
16 plt.ylabel("n_eff")
17 plt.title("Diagrama de Dispersao (TE 1D)")
18 plt.grid(True)
19 plt.legend()
20 plt.savefig("03.png")
21 plt.show()
```

Listing 3: Script de pós-processamento: visualização dos modos guiados e diagrama ilustrativo de dispersão.

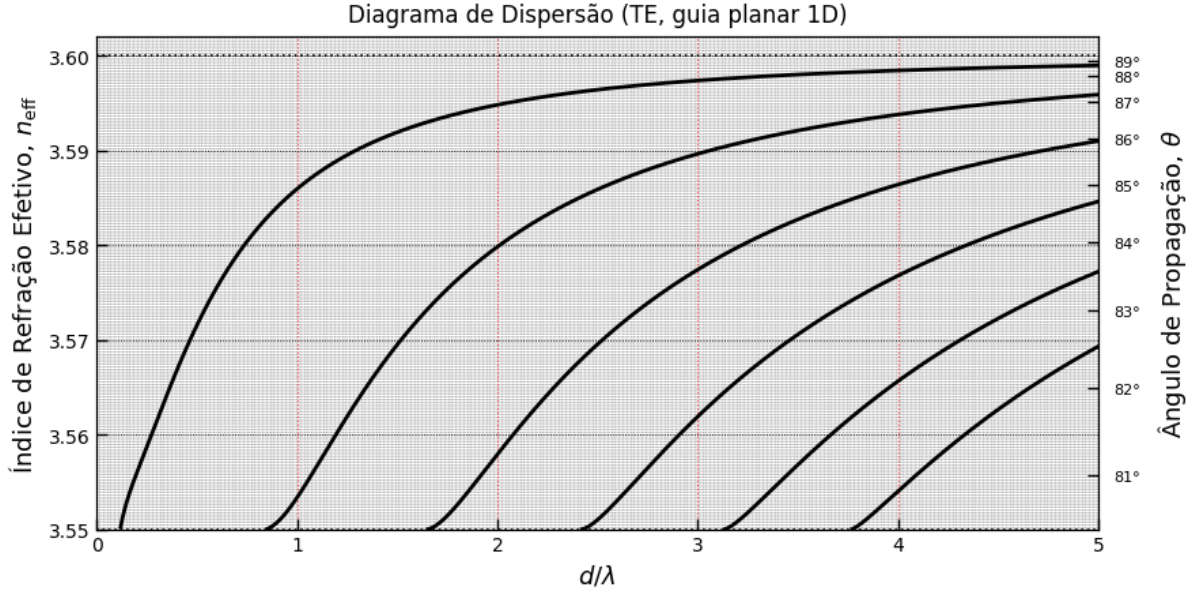


Figura 3: Diagrama de dispersão ilustrativo (n_{eff} em função de d/λ). O ponto $d/\lambda = 1.0$ está destacado para o caso de validação considerado.

4 Conclusão

O trabalho demonstrou a **implementação computacional** da Análise Modal 1D (Modos TE) utilizando o Método das Diferenças Finitas em Python. Ao estruturar o estudo nas etapas de **Pré-processamento, Processamento e Pós-processamento**, foi possível simular o guia planar, montar as matrizes **A** e **B** associadas ao problema de autovalores e, de forma consistente, filtrar os resultados para obter apenas os modos fisicamente guiados ($n_{\text{clad}} < n_{\text{eff}} < n_{\text{core}}$). A validação visual e numérica confirma que a lógica implementada está correta e atende aos requisitos da disciplina.

Referências

- Andres Pablo Lopez Barbero. Fotônica computacional: Análise modal e bpm. Notas de aula (Disciplina TCE11209), UFF, 2025.
- Katsunari Okamoto. *Fundamentals of Optical Waveguides*. Elsevier, 3 edition, 2021.
- Allan W. Snyder and John D. Love. *Optical Waveguide Theory*. Springer, 1983.